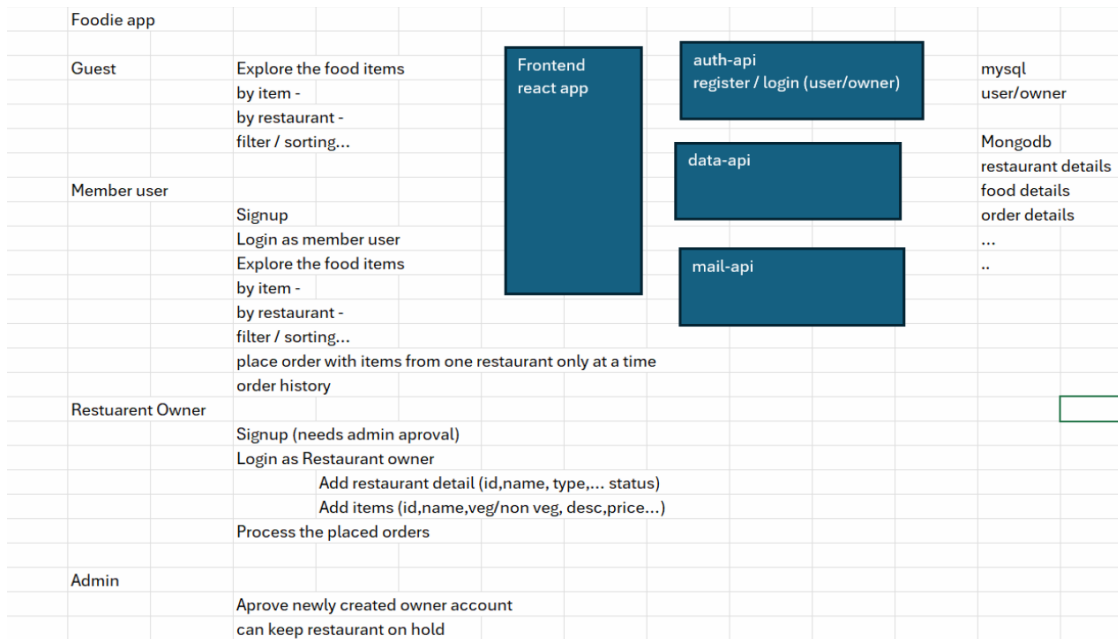


Foodie App (Zomato-like)



Roles

- 1-Guest User
- 2-Registered User
- 3-Restaurant Owner
- 4-Admin

1-Guest User

View restaurants
View food items
Filter/sort/Search/Cetegory
View food details (ChatGPT powered/gemini)
Cannot place order

Restrictions

Order button disabled
Checkout redirects to Login

2-Registered User

Authentication

Signup (Email + OTP verification)
Login / Logout
Forgot password

Functionalities

Browse restaurants (within 10km)
Browse food items
Filter (veg/non-veg, price, rating)
Order food (only 1 restaurant at a time)
Track order
Order history
Membership purchase
Priority order (if member)

3-Restaurant Owner

Authentication

Signup → Admin Approval Required
Log in after approval
Forgot password

Functionalities

Add / edit restaurant profile
Add/Edit/Delete food items
Manage availability
View incoming orders
Set preparation time
Process orders (Accept / Reject / Preparing / Dispatched)

4-Admin

Approve / reject restaurant owner
Activate / deactivate restaurants
Put restaurant “ON HOLD”
View system analytics
Manage memberships

Tech Stack

Frontend

React

Backend APIs

Spring Boot (Microservices)

Database

MySQL for StructuredDb

MongoDB for NonStructuredDb

Maps

Google Maps API

OpenAI

Gemini API Integration for Nutrition details

Mail

Mail API

Auth

Mail API

Core Modules

Authentication & Authorization
Restaurant Management
Food / Menu Management

Order Management
Membership & Priority Handling
Location & Distance Validation
ChatGPT Food Intelligence
Notification (Email OTP, Order updates)

DETAILED MODULE DESIGN

AUTH SERVICE (Spring Boot + MySQL)

The Auth Service is responsible for identity, security, and access control in your Foodie App.

It answers questions like:

- Who is the user?
- Are they verified?
- What role do they have?
- Are they allowed to access this API?

APIs – What Each Endpoint Does

POST /auth/signup

Purpose: Register a new user (User / Restaurant Owner)

What happens internally:

- User submits email, password, role
- *Password is hashed
- User record is saved in users
- Role is mapped in user_roles
- OTP is generated and saved
- OTP email is sent
- User is NOT active yet until OTP is verified.

Note: first, we take the email, then verify. After that, we open the option for other details

POST /auth/login

Purpose: Authenticate user and issue JWT token

What happens internally:

Email & password are validated
Password hash is matched
User status is checked (verified / active)
JWT access token + refresh token generated
Tokens returned to frontend

JWT is used in all protected APIs.

POST /auth/verify-otp

Purpose: Verify email using OTP

What happens internally:

User submits OTP
OTP is matched from otp_verification
OTP expiry is checked
User account is marked as verified
OTP record is removed or expired

This step prevents fake registrations.

POST /auth/forgot-password

Purpose: Start password reset flow

What happens internally:

User submits email
*OTP or reset token is generated
Token stored in DB
Reset link / OTP sent via email

POST /auth/reset-password

Purpose: Set a new password

What happens internally:

Token / OTP is validated
New password is hashed
password is updated in users
Old tokens are invalidated

Improves account security.

POST /auth/refresh-token

Purpose: Generate new access token without re-login

Why needed:

Access tokens are short-lived

Refresh token creates a new access token

What happens internally:

Refresh token is validated

New JWT access token is generated

Old access token expires

Used for smooth user experience.

Database Name

foodie_auth_db → Authentication & users
foodie_business_db → Restaurants, food, orders

Reason: Auth data = sensitive & structured Business data = large & flexible

How Many Databases

Database	Type	Purpose
foodie_auth_db	MySQL	Users, roles, login, security
foodie_business_db	MongoDB	Restaurants, food, orders

This database handles **WHO** the user is.

Table 1: USERS

user_id (PK)

name

email (UNIQUE)

password_hash
profile_image_url
is_verified
status
created_at

Notes

No role stored here
Password is hashed
Admin, user, owner all live here

Table 2: ROLES

Stores predefined system roles (static).

role_id (PK)
role_name

1 -> ADMIN
2 -> USER
3 -> RESTAURANT_OWNER

Inserted **once** when app is created.

Table 3: USER_ROLES

Connects users to roles (THIS is the key table).

user_role_id (PK)
user_id (FK → USERS)
role_id (FK → ROLES)

Why Needed?

One user can have multiple roles
Clean RBAC implementation

Table 4: OTP_VERIFICATION

Handles OTP for signup & password reset.

otp_id (PK)
user_id (FK → USERS)
otp
purpose
expiry_time

Table 5: MEMBERSHIP optional

Stores premium membership info.

membership_id (PK)
user_id (FK → USERS)
type
start_date
end_date
status

Tables in **foodie_business_db** (MongoDB)

This database handles WHAT business data exists.a

Collection 1: RESTAURANTS

Stores restaurant details.

restaurantId
ownerUserId
name
description
latitude
longitude
status
prepTime
createdAt

ownerUserId connects to USERS.user_id

Collection 2: FOOD_ITEMS

Stores food/menu items.

foodId
restaurantId
name
price
vegFlag
description
imageUrls
availability

ownerUserId connects to USERS.user_id

Collection 3: ORDERS

Stores order lifecycle.

orderId
userId
restaurantId
items[]
totalAmount
orderStatus
priority
deliveryDistance
createdAt

Collection 4: ADDRESSES

Stores order lifecycle.

addressId
userId

latitude
longitude
fullAddress

AUTH DATABASE CONNECTIONS (Summary)

The **USERS** table stores basic user identity and authentication details.

The **ROLES** table stores predefined system roles (ADMIN, USER, RESTAURANT_OWNER).

The **USER_ROLES** table maps users to roles, enabling role-based access control.

The **OTP_VERIFICATION** table is linked to USERS for email verification and password reset.

The **MEMBERSHIP** table is linked to USERS to store premium membership details.

Key Point

A user's permissions are determined only by entries in the USER_ROLES table.

BUSINESS DATABASE CONNECTIONS (Summary)

The **RESTAURANTS** collection stores restaurant details and is linked to the restaurant owner using ownerUserId.

The **FOOD_ITEMS** collection is linked to RESTAURANTS using restaurantId, representing the menu of each restaurant.

The **ORDERS** collection links customers and restaurants using userId and restaurantId.

The **ADDRESSES** collection stores user delivery locations and is linked to USERS using userId.

Key Points

Business data is stored in **MongoDB** for flexibility and scalability.

Connections between MySQL and MongoDB are maintained using IDs, not foreign keys.

This design supports high read/write traffic and easy scaling.

The system uses MySQL for authentication and authorization data and MongoDB for business data. User identity is stored in the USERS table, while roles are defined in the ROLES table and mapped using the USER_ROLES table. Business entities such as restaurants, food items, and orders are stored in MongoDB and linked to users using user IDs. This separation ensures scalability, security, and clean system design.

AUTH SERVICE (Spring Boot + MySQL)

```
POST /auth/signup
POST /auth/login
POST /auth/verify-otp
POST /auth/forgot-password
POST /auth/reset-password
POST /auth/refresh-token
```

USER SERVICE

```
GET /users/profile
PUT /users/profile
GET /users/orders
POST /users/membership/upgrade
```

RESTAURANT SERVICE (MongoDB)

```
POST /restaurants
PUT /restaurants/{id}
GET /restaurants/nearby
GET /restaurants/{id}
```

FOOD / MENU SERVICE (MongoDB)

```
POST /foods
PUT /foods/{id}
DELETE /foods/{id}
GET /foods/restaurant/{restaurantId}
GET /foods/{foodId}
```

ORDER SERVICE (MongoDB)

```
POST /orders
GET /orders/{orderId}
GET /orders/user
GET /orders/restaurant
PUT /orders/{id}/status
```

MEMBERSHIP SERVICE

```
POST /membership/subscribe
GET /membership/status
```

Logic

- Membership = priority flag
- Higher queue priority
- Faster processing SLA

CHATGPT FOOD INTELLIGENCE SERVICE

Trigger

User clicks on food item → frontend calls ChatGPT API

Provide:

- Nutrition facts
- Ingredients
- Health benefits
- Image gallery idea
- Best time to eat

POST /ai/food-info

GOOGLE MAPS INTEGRATION

Logic

Capture user location

Fetch nearby restaurants ($\leq 10\text{km}$)

Validate before order placement

GET /restaurants/nearby?lat=&lng=

USER SIGNUP FLOW

User → Signup

→ Enter Email

→ Send OTP

→ Verify OTP

→ Account Activated

ORDER PLACEMENT FLOW

User selects restaurant

→ Check distance ($\leq 10\text{km}$?)

→ Select food items

→ Membership?

YES → High Priority

NO → Normal Priority

→ Place Order

→ Notify Restaurant

RESTAURANT APPROVAL FLOW

Owner Signup

→ Admin Review

→ Approve?

YES → Activate

NO → Reject

CHATGPT FOOD INFO FLOW

User clicks food

→ Backend sends prompt to ChatGPT

→ Response received

→ Cache response

→ Show details

EXTRA REAL-WORLD FEATURES (RECOMMENDED)

Order cancellation window

Refund handling

Reviews & ratings

Offers & coupons

Scheduled orders

Admin analytics

Delivery partner integration (future)